DESENVOLVIMENTO BACK-END COM JAVA

Desenvolvimento de Serviços Web e Testes com Java

Do código que funciona ao código que podemos testar e disponibilizar para outros sistemas.



Programação

Transformar problemas em soluções com Java.

Qualidade

Verificar automaticamente se o código se comporta como deveria.

Integração

Consumir e desenvolver serviços web para que nossas aplicações se comuniquem com outros sistemas.

NOSSO OBJETIVO

Desenvolver aplicações Java que não apenas funcionem, mas que possam ser **testadas, integradas e evoluídas com segurança.**

DESENVOLVIMENTO BACK-END COM JAVA

O que vamos aprender?

As três competências da disciplina

Testes, integração e desenvolviimento de APIs para aplicações Java mais confiáveis e conectadas.



Testes unitários com JUnit

Criar testes automatizados para verificar resultados, regras de negócio e tratamento de erros.

Consumo de serviços web

Fazer requisições HTTP com Java e utilizar dados vindos de APIs RESTful.

Desenvolvimento de serviços web

Criar APIs RESTful com Java e Javalin para disponibilizar funcionalidades a outras aplicações.

AO FINAL DO TRIMESTRE

Você deverá conseguir **testar comportamentos, consumir APIs e desenvolver seus próprios serviços web** com Java.

DESENVOLVIMENTO BACK-END COM JAVA

Como a disciplina está organizada?

Nove etapas, três grandes blocos de aprendizagem

A disciplina foi organizada como uma jornada progressiva: testar, integrar e desenvolver serviços web com Java.



Bloco 1 — Testes automatizados

Semanas 1 a 4

1. Introdução ao JUnit
2. Configuração e testes unitários
3. Requisitos e testes isolados
4. Dublês de teste e helpers

Bloco 2 — Comunicação HTTP

Semanas 5 e 6

5. REST, HTTP e HTTPS
6. Requisições GET e POST

Bloco 3 — Desenvolvimento de APIs

Semanas 7 a 9

7. Javalin
8. HTTP Handlers
9. Testes em Javalin

VISÃO GERAL DA JORNADA

Primeiro **testamos** o código, depois **consumimos serviços** e, por fim, **desenvolvemos nossas próprias APIs** com Java.

DESENVOLVIMENTO BACK-END COM JAVA

Quando teremos cada etapa?

Calendário das aulas

Quintas e sextas
20h30 às 22h00

Nossa jornada de aprendizagem entre outubro e dezembro, com 9 semanas de conteúdo, 1 semana de alinhamento final e 1 semana de reentrega.



OUTUBRO

1	**Semana 1** 08 e 09/10	Introdução ao JUnit
2	**Semana 2** 15 e 16/10	Configuração e testes unitários
3	**Semana 3** 22 e 23/10	Requisitos e testes isolados
4	**Semana 4** 29 e 30/10	Dublês de teste e helpers

NOVEMBRO

5	**Semana 5** 05 e 06/11	REST, HTTP e HTTPS
6	**Semana 6** 12 e 13/11	Requisições GET e POST
7	**Semana 7** 19/11	Javalin
	20/11	**Feriado nacional** (sem aula e sem reposição)
8	**Semana 8** 26 e 27/11	HTTP Handlers

DEZEMBRO

9	**Semana 9** 03 e 04/12	Testes em Javalin
10	**Semana 10** 10 e 11/12	Alinhamento final
11	**Semana 11** 17 e 18/12	Reentrega de trabalhos

VISÃO GERAL DO TRIMESTRE

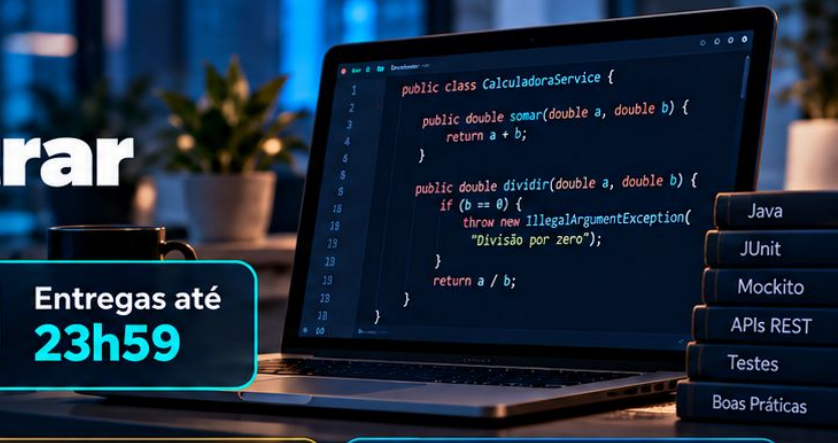
As **9 primeiras semanas** desenvolvem o conteúdo da disciplina.
As **2 últimas** apoiam o **fechamento e a reentrega dos trabalhos**.

DESENVOLVIMENTO BACK-END COM JAVA

Como vamos demonstrar nossa evolução?

Avaliações e prazos

Quatro avaliações ao longo do trimestre para acompanhar sua evolução na disciplina.



Entregas até 23h59

1 TP1

26/10

Testes unitários com **JUnit** sobre uma calculadora científica.

2 TP2

09/11

Reembolso médico com regras de negócio, **JUnit, Mockito** e dublês de teste.

3 TP3

23/11

Enunciado será disponibilizado futuramente.

4 Assessment

08/12

Trabalho final da disciplina. Enunciado será disponibilizado futuramente.

Importante: nos enunciados do TP1 e do TP2, o uso de **IA generativa é proibido.**

VISÃO GERAL DAS AVALIAÇÕES

Aprender é construir conhecimento. Avaliar é demonstrar o que você consegue fazer com ele.

DESENVOLVIMENTO BACK-END COM JAVA

Como vamos trabalhar?

Aprender, construir, testar e evoluir

Nossa dinâmica une teoria, prática e evolução contínua para transformar conhecimento em competência.



Aprender

Compreender conceitos, problemas e requisitos.

Construir

Transformar ideias em soluções com Java.

Testar

Verificar comportamentos e tratar erros.

Evoluir

Melhorar o código com mais segurança.

A prática transforma conhecimento em competência.
Os testes nos ajudam a **evoluir com segurança.**

DESENVOLVIMENTO BACK-END COM JAVA

Nossa evolução com Java

De instruções simples a sistemas orientados a objetos

No trimestre passado, saímos dos primeiros programas com variáveis e decisões e chegamos a aplicações com classes, objetos, herança, polimorfismo e organização de responsabilidades.





Fundamentos

Variáveis, operações, decisões e repetição.

Interação

Entrada de dados, métodos e reutilização.

Orientação a objetos

Classes, objetos, construtores e encapsulamento.

Modelagem e organização

Herança, polimorfismo, interfaces, exceções e arquivos.

VISÃO GERAL DA EVOLUÇÃO

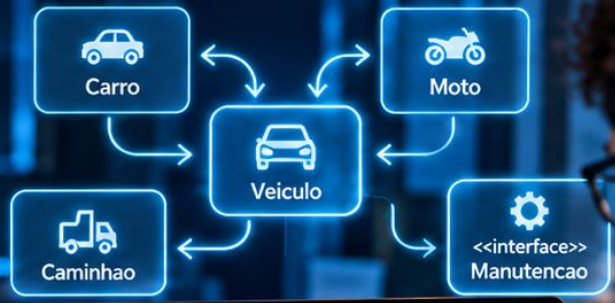
Evoluímos de **instruções simples** para **aplicações organizadas**.
Agora, vamos aprender a **garantir a qualidade** do que construímos.

DESENVOLVIMENTO BACK-END COM JAVA

O que conseguimos construir?

Revisitando os projetos do trimestre passado

Dos primeiros exercícios a aplicações com regras de negócio, desenvolvemos projetos que uniram lógica, orientação a objetos e organização de responsabilidades.





FundamentosJava
Lógica, entrada de dados, decisões e repetições.

Fundamentospoo
Classes, objetos, construtores e encapsulamento.

GaragemService
Herança, polimorfismo, interfaces, exceções e arquivos.

integracao
Eventos, inscrições e regras de negócio.

VISÃO GERAL DOS PROJETOS

Aprendemos a construir funcionalidades. **Agora, precisamos aprender a verificar seus comportamentos.**

DESENVOLVIMENTO BACK-END COM JAVA

Como verificávamos nosso código?

Do teste manual aos testes automatizados

No trimestre passado, executávamos o programa e observávamos os resultados. Agora, vamos aprender a verificar comportamentos de forma automática e repetível.

Verificação manual

```
C:\projetos> java EventoApp
Nome: Conferência Java
Data: 15/04/2025
Preço: 250.00

Evento criado com sucesso!
```

- Executar o programa
- Observar o resultado
- Conferir manualmente

Verificação automatizada

JUnit

Testes executados: 5 | Aprovados: 4 | Falhas: 1

- EventoTest
  - deveCriarEventoComDadosValidos()
  - deveCalcularDesconto()
  - deveValidarPrecoNegativo()
  - deveFormatarData()
  - deveRetornarNome()

- Executar testes
- Comparar resultado esperado
- Identificar falhas rapidamente

EventoTest.java

```
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

class EventoTest {

    @Test
    void deveCalcularDesconto() {
        Evento evento = new Evento("Conferência Java", 250.0);
        double desconto = evento.calcularDesconto(20);
        assertEquals(200.0, desconto, 0.01);
    }

    @Test
    void deveValidarPrecoNegativo() {
        assertThrows(IllegalArgumentException.class, ()
                -> new Evento("Evento", -10.0));
    }
}
```

VISÃO GERAL DA MUDANÇA

Testar é comparar o comportamento observado com o **comportamento esperado.**